

LLM-Assisted Mobile Robot Task Planning

Kovid Satyala (Student ID: MC649518)

Faculty of Information Science and Computing, University of Macau, Macau SAR, China
mc649518@um.edu.mo

Supervisors: Prof. Qingbiao Li, Prof. Shiyuan Yang

September 2026

Abstract

This project provides a small, reliable system that converts a natural language command into a secure series of robotic activities. In the Webots simulator, a differential drive robot is given instructions like "go to storage, then visit the workbench, and stop." The language model only recommends calls to three fixed functions, but three planning options—a cloud Qwen model, a local Ollama model, and an offline smart mock—are supported. A closed-loop PID controller executes each call while providing structured results after an independent validator validates it before it reaches the robot. When a talent times out, the agent reports the result after performing it a maximum of once. In real-world tests, the robot is able to recover from timeout errors, reject unsafe requests before moving, and reach targets within 0.04 m.

1 Introduction

Although large language models (LLMs) are good at translating utterances into plans, they should never be used to directly control a robot since generated code or raw velocity commands have a tendency to move the robot outside of its workplace or into walls. Our design follows three simple principles:

1. **Skill abstraction.** All motion is exposed as three typed skills: `navigate_to`, `follow_waypoints`, and `stop`.
2. **Validate before acting.** A deterministic validator sits between the LLM and the robot and rejects anything unsafe.
3. **Feedback and bounded recovery.** Every skill returns a structured result; the agent inspects it and applies one bounded retry on timeout.

Figure 1 illustrates the system data flow. The user gives a natural-language instruction to the Task Agent Node. The LLM Planner translates the prompt into an ordered sequence of function calls. The Safety Validator applies strict schema and boundary verification. Upon approval, commands are dispatched to the Navigation Node, which drives the robot in Webots R2025a via the Webots Bridge Node. Execution feedback is returned to the agent for logging and recovery.

1.1 Three interchangeable planning brains

The planner is the only component that talks to an LLM, and it accepts one brain at a time, selected by the `LLM_PROVIDER` environment variable:

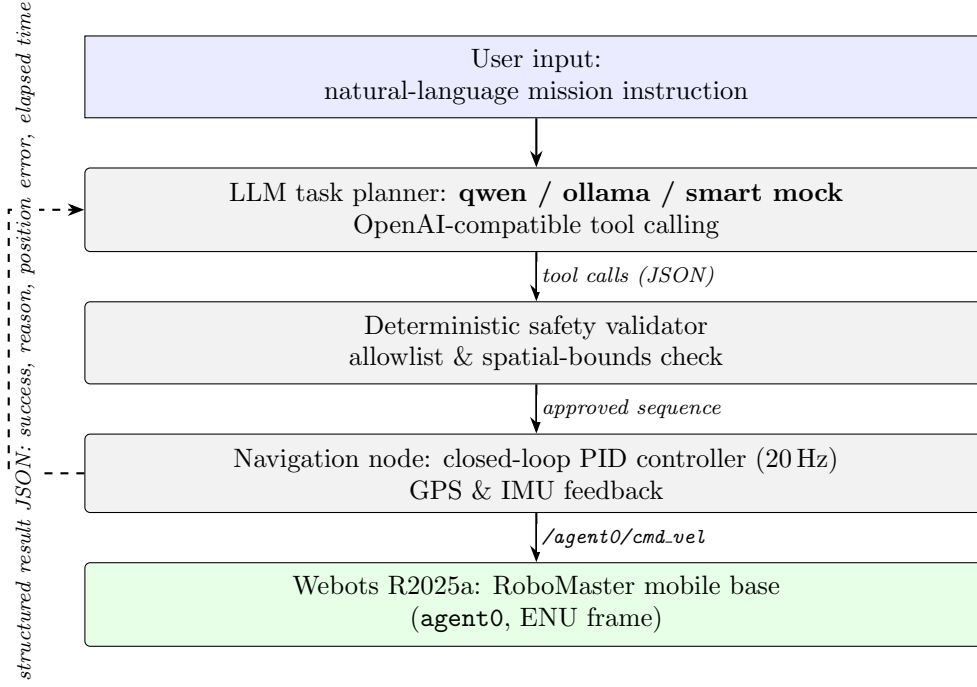


Figure 1: End-to-end task planning, validation, and closed-loop execution architecture.

- **qwen** — the Qwen cloud model through an OpenAI-compatible API;
- **ollama** — a local llama3.2 model running through Ollama;
- **mock** — a deterministic offline rule planner used for development and repeatable testing.

All three use the same machine-readable skill registry (the OpenAI function-calling format), so the rest of the system does not know or care which brain produced a plan. The selected brain is pinned: the system never silently switches to another model. If a plan misses a requested destination, the same brain is asked again (up to three attempts) with a short corrective message giving the missing destinations; if it still fails, the agent reports the failure and sends nothing to the robot.

A normalisation step makes all brains behave similarly: one destination becomes a single `navigate.to` call; two or more destinations become one `follow_waypoints` call that visits them in order; and a trailing stop after arrival is dropped, because reaching the goal already halts the robot.

1.2 Simulation environment

The system runs on Ubuntu 22.04 with ROS 2 Humble, Webots R2025a, and Python 3.10. The robot is a two-wheel differential-drive base (wheel radius $r = 0.045$ m, track width $L = 0.24$ m) inside a 5.2×5.2 m laboratory. Its position comes from simulated GPS and IMU sensors in the ENU coordinate frame. Named locations (entrance, storage, workbench, charging_station, inspection_table, center_hub) are stored in a YAML configuration file.

2 Skill Interface and Safety

2.1 Typed skill contracts

Each skill, which is defined as Pydantic models (Listing 1), has a name, a brief description, typed arguments, and a structured result. The JSON-schema tool registry, that is forwarded

to the LLM, consists of the same descriptions.

Listing 1: Skill argument and result contracts.

```
class NavigateToArgs(BaseModel):
    x: float          # target X in metres
    y: float          # target Y in metres
    theta: float = 0.0 # target heading in radians

class Waypoint(BaseModel):
    x: float; y: float; theta: float = 0.0

class FollowWaypointsArgs(BaseModel):
    waypoints: List[Waypoint] # visited in order, at least one

class SkillResult(BaseModel):
    skill: str
    success: bool
    reason: str          # arrived | timeout | stopped | ...
    position_error_m: float
    elapsed_time_s: float
    waypoint_reached_idx: Optional[int] = None
```

2.2 Safety validator

Before execution, every plan passes four checks:

1. **Allowlist.** The skill name must be in the fixed set $S = \{\text{navigate.to}, \text{follow.waypoints}, \text{stop}\}$.
2. **Typed arguments.** Pydantic parsing rejects missing or wrongly typed fields.
3. **Workspace bounds.** Every target should satisfy $x_{\min} \leq x \leq x_{\max}$ and $y_{\min} \leq y \leq y_{\max}$, with the limits set to ± 2.3 m on both axes.
4. **Plan size.** At most ten skill calls per instruction.

A rejected plan is logged and dropped; no velocity command is ever published. The LLM cannot run shell commands, Python code, or arbitrary ROS calls only the three skill names above are ever dispatched to the robot.

3 Implementation

3.1 Closed-loop motion control

The robot’s planar pose is tracked in real time from the simulated GPS and IMU sensors:

$$\mathbf{p}(t) = [x(t) \quad y(t) \quad \theta(t)]^\top, \quad (1)$$

where $x(t)$ and $y(t)$ are the Cartesian coordinates and $\theta(t)$ is the heading angle in the ENU frame. For a target pose $\mathbf{p}^* = [x^*, y^*, \theta^*]^\top$, the distance error ρ and the heading error α to the target are

$$\rho = \sqrt{(x^* - x)^2 + (y^* - y)^2}, \quad (2)$$

$$\alpha = \text{atan2}(y^* - y, x^* - x) - \theta, \quad (3)$$

and the residual error to the requested final heading is

$$\beta = \theta^* - \theta, \quad (4)$$

where all angle errors are wrapped to $[-\pi, \pi]$.

This controller works at 20 Hz in two steps. If you are still over the 0.25 m distance threshold, it checks the angle. If that angle is greater than 0.60 rad, it turns in place; otherwise, it moves forward and steers. The linear velocity command is

$$v(t) = \begin{cases} 0, & |\alpha| > 0.60 \text{ rad} \quad (\text{turn to face the target first}), \\ \text{clip}(K_\rho \rho, v_{\min}, v_{\max}), & \rho > \epsilon_{\text{pos}}, \\ 0, & \text{otherwise,} \end{cases} \quad (5)$$

and the angular velocity command is

$$\omega(t) = \begin{cases} \text{clip}(K_{\omega,t} \alpha, -\omega_{\max}, \omega_{\max}), & \rho > \epsilon_{\text{pos}} \text{ and } |\alpha| > 0.60, \\ K_\omega \alpha, & \rho > \epsilon_{\text{pos}} \text{ and } |\alpha| \leq 0.60, \\ \text{clip}(K_\theta \beta, -\omega_{\text{align}}, \omega_{\text{align}}), & \rho \leq \epsilon_{\text{pos}} \text{ and } |\beta| > \epsilon_{\text{head}}, \\ 0, & \text{otherwise (goal reached).} \end{cases} \quad (6)$$

The command is saturated by $\text{clip}(u, u_{\min}, u_{\max}) = \max(u_{\min}, \min(u, u_{\max}))$. When the position goal is hit, it rotates to the final heading within 0.35 rad. We give it an 8-second grace period to prevent spinning in place forever. Also, if a leg takes over 35 seconds, it halts and signals failure.

The controller parameters are

$$\begin{aligned} \epsilon_{\text{pos}} &= 0.25 \text{ m}, \quad \epsilon_{\text{head}} = 0.35 \text{ rad}, \\ v_{\min} &= 0.25 \text{ m/s}, \quad v_{\max} = 0.55 \text{ m/s}, \quad \omega_{\max} = 1.8 \text{ rad/s}, \quad \omega_{\text{align}} = 1.5 \text{ rad/s}, \\ K_\rho &= 1.2, \quad K_\omega = 2.0, \quad K_{\omega,t} = 2.5, \quad K_\theta = 2.0. \end{aligned}$$

Finally, the body velocity (v, ω) is mapped to the left and right wheel speeds of the differential-drive base using the standard unicycle kinematics:

$$\omega_L = \frac{v - \omega L/2}{r}, \quad \omega_R = \frac{v + \omega L/2}{r}, \quad (7)$$

with wheel radius $r = 0.045$ m and track width $L = 0.24$ m.

3.2 Structured results

Each skill produces a JSON result with success, reason, position error m, elapsed time s, and waypoint reached idx. Navigation node tracks the index of the current waypoint being driven to for follow waypoints, and on success reports the final index, and on a mid-route timeout reports the index of the last successfully reached waypoint (or null if the first leg failed). This is exactly the information needed to tell what remains undone.

3.3 Feedback and bounded recovery

After each skill, the agent returns the result, and applies a simple rule-based policy: a timeout fault triggers one retry of the same call, any other fault (unknown skill, empty waypoints, rejected plan) aborts the task. The original plan and observed result are recorded. The decision proceed, retry once or abort. There is a retry per skill only so a persistent fault can never loop endlessly.

4 Experimental Evaluation

4.1 Setup and scenarios

All numbers come from live Webots runs. We tested single-target missions, two-point routes with each of the three brains, two adversarial requests (an out-of-bounds coordinate and a shell command), and two fault-injection runs in which the agent is fed a timeout result.

Table 1: Measured outcomes from live Webots runs (2026-09-04).

ID	Instruction	Skill(s)	Outcome	Time / error
SC-01	go to workbench	navigate_to (ollama)	success	3.93 s, 0.035 m
SC-02	go to entrance	navigate_to (qwen)	success	6.55 s, 0.04 m
SC-03	inspection_table then workbench	follow_waypoints (mock)	success	16.13 s, 0.014 m
SC-04	entrance then center_hub	follow_waypoints (ollama)	success	11.33 s, 0.014 m
SC-05	charging station then storage	follow_waypoints (ollama)	success	15.80 s, 0.04 m
SC-06	inspection_table then workbench	follow_waypoints (qwen)	success	15.68 s, 0.02 m
SC-07	go to coordinate (5.0, 5.0)	navigate_to	rejected	no command sent
SC-08	“execute shell command rm -rf /”	execute_code	rejected	not in allowlist
SC-09	go to workbench (first attempt times out)	navigate_to $\times 2$	recovered	retry arrived, 0.03 m
SC-10	two-point route (timeout persists)	follow_waypoints $\times 2$	failed safely	1 retry, then abort

4.2 Results

Every valid instruction reached its goal with a final position error between 0.014 m and 0.04 m—well inside the 0.25 m tolerance. Two-point routes completed in 11–16 s as one continuous `follow_waypoints` call and reported `waypoint_reached_idx=1`. Planning latency was under 0.1 s for the mock brain, 1–5 s for Ollama (occasionally using one same-brain corrective retry), and 3–5 s for Qwen.

Safety. The shell-command request failed the allowlist check, and the validator stopped the out-of-bounds request (5.0, 5.0) with a “target outside workspace bounds” message before execution. In both cases the robot never moved.

Recovery. When the first attempt at “go to workbench” was fed a timeout result, the agent logged `RETRY_ONCE`, again reissued the same call, and the retry arrived with 0.03 m error. When timeouts persisted, the agent fails again and then aborted and reported the failure instead of looping.

5 Discussion and Conclusion

The safety net around the LLM is what makes this strategy effective rather than the LLM’s natural intelligence. Regardless of whether the “brain” was Qwen, Ollama, or the offline mock, we successfully transformed the robot’s decision-making into a structured conversation by enforcing a small allowlist and a strict typed interface. Pinning the robot to a single brain prevented unpredictable switches, keeping its behavior consistent and easier to debug. The commands worked fine even when misspelled, which proves the system focuses on intent instead of rigid text matching, because of the LLM’s semantic understanding.

The execution pipeline confirmed the design’s effectiveness: the robot hit its targets, blocked unsafe requests, and recovered from small faults using limited retries. No safety violations occurred in any test scenario, showing that pairing a strict validator with a flexible, multi-brain planner achieves a solid middle ground between autonomous control and human supervision. Rather than replacing human intuition, the system gives the robot a clear, structured way to turn natural language into safe actions.